



Digital Integrated Circuit Design

Lecture-6: Multiplication

Chih-Feng Wu (吳志峰), PhD.

*Department of Electronics Engineering, Chang Gung University,
Taoyuan, Taiwan (ROC)*

May 22, 2026

Chih-Feng Wu



Outline

- ☐ Shift/Add Multiplication Algorithm
- ☐ Basic Hardware Multiplication
- ☐ Multiplication of Signed Numbers
- ☐ Radix-4 Multiplier
- ☐ Modified Booth's Recoding
- ☐ Full-Tree Multiplier
- ☐ Array Multiplier

Chih-Feng Wu

Shift/Add Multiplication Algorithm

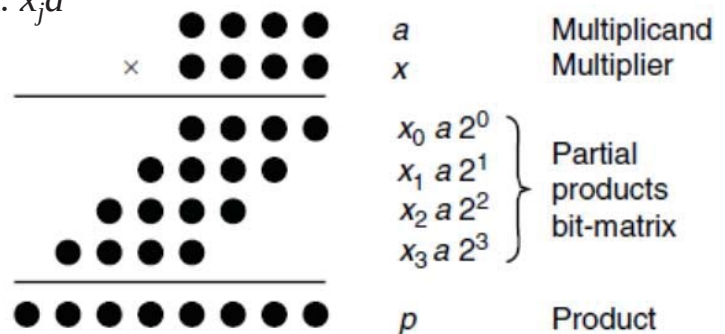
Shift/Add Multiplication

Notation for our discussion of multiplication algorithms:

- a : Multiplicand $a_{k-1}a_{k-2} \dots a_1a_0$
- x : Multiplier $x_{k-1}x_{k-2} \dots x_1x_0$
- p : Product ($a \times x$) $p_{2k-1}p_{2k-2} \dots p_3p_2p_1p_0$

First, we consider unsigned operands.

- Partial product term $p^{(j)}$: $x_j a$



Using Two-Operand Adder (cont'd)

Multiplication with **right shifts**: top-to-bottom accumulation

$$p^{(j+1)} = (p^{(j)} + x_j a 2^k) 2^{-1}$$

| —add— |

| —shift right— |

$$p^{(0)} = 0 \text{ and } p^{(k)} = p$$

$$\text{After } k \text{ iterations: } p^{(k)} = p = ax + p^{(0)}2^{-k}$$

Multiply-add operation

- The expression $ax + y$ is evaluated when $p^{(0)}$ is initialized to $y2^k$.

(a) Right-shift algorithm

a	1	0	1	0				
x	1	0	1	1				
$p^{(0)}$	0	0	0	0				
$+x_0a$	1	0	1	0				
$2p^{(1)}$	0	1	0	1	0			
$p^{(1)}$	0	1	0	1	0			
$+x_1a$	1	0	1	0				
$2p^{(2)}$	0	1	1	1	1	0		
$p^{(2)}$	0	1	1	1	1	0		
$+x_2a$	0	0	0	0				
$2p^{(3)}$	0	0	1	1	1	1	0	
$p^{(3)}$	0	0	1	1	1	1	0	
$+x_3a$	1	0	1	0				
$2p^{(4)}$	0	1	1	0	1	1	1	0
$p^{(4)}$	0	1	1	0	1	1	1	0

Using Two-Operand Adder (cont'd)

□ Multiplication with **left shifts**: bottom-to-top accumulation

$$\blacksquare p^{(j+1)} = 2p^{(j)} + x_{k-j-1}a$$

|shift left|

|——— add ———|

$$\blacksquare p^{(0)} = 0 \text{ and } p^{(k)} = p$$

$$\blacksquare \text{After } k \text{ iterations: } p^{(k)} = p = ax + p^{(0)}2^k$$

□ Multiply-add operation

- The expression $ax + y$ is evaluated if we initialize $p^{(0)}$ to $y2^{-k}$.

(b) Left-shift algorithm

a		1	0	1	0
x		1	0	1	1
$p^{(0)}$		0	0	0	0
$2p^{(0)}$	0	0	0	0	0
$+x_3a$		1	0	1	0
$p^{(1)}$	0	1	0	1	0
$2p^{(1)}$	0	1	0	1	0
$+x_2a$		0	0	0	0
$p^{(2)}$	0	1	0	1	0
$2p^{(2)}$	0	1	0	1	0
$+x_1a$		1	0	1	0
$p^{(3)}$	0	1	1	0	0
$2p^{(3)}$	0	1	1	0	0
$+x_0a$		1	0	1	0
$p^{(4)}$	0	1	1	0	1

Chih-Feng Wu

5

Basic Hardware Multipliers

□ Hardware realization of the multiplication algorithm with right-shift

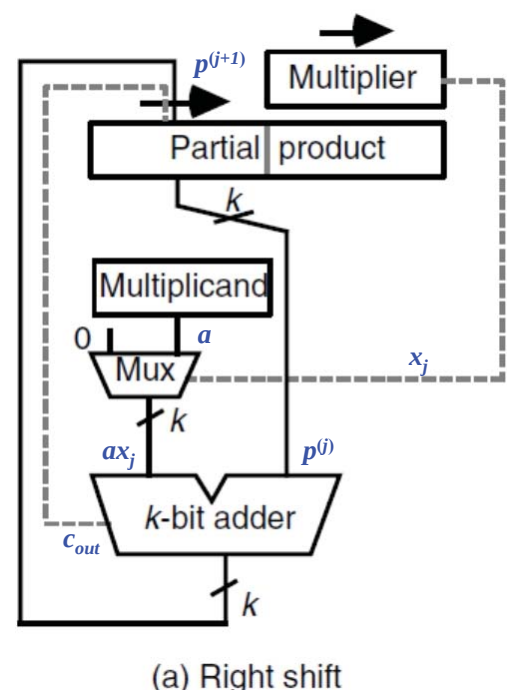
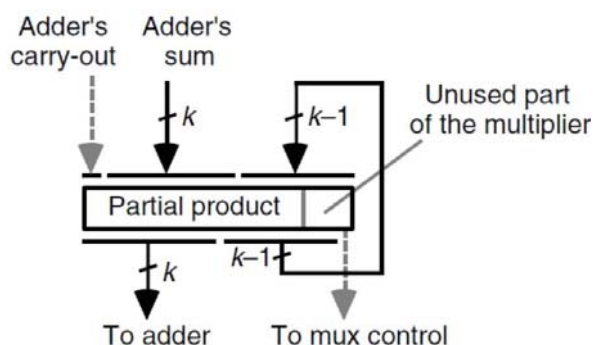
$$p^{(j+1)} = (p^{(j)} + x_j a 2^k) 2^{-1}$$

|———add———|

|———shift right———|

□ Double-width register holds

■ Partial product and used multiplier



(a) Right shift

Chih-Feng Wu

6

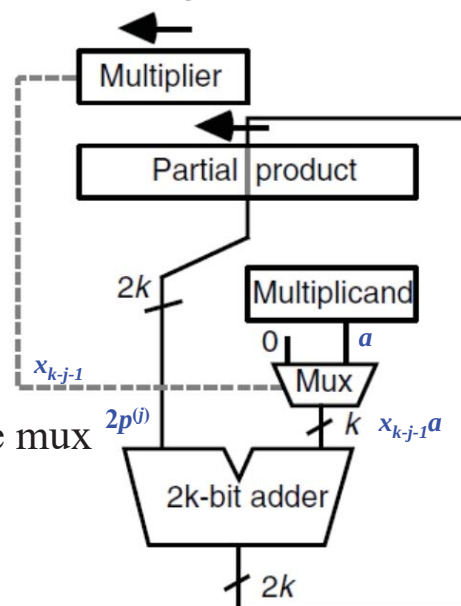
Basic Hardware Multipliers (cont'd)

- Hardware realization of the multiplication algorithm with left-shift

$$p^{(j+1)} = \underbrace{2p^{(j)}}_{\text{shift left}} + \underbrace{x_{k-j-1} a}_{\text{add}}$$

- For (a) and (b)

- Use a set of k AND gates to replace the mux
- One input for all ANDs is tied to x_j .



(b) Left shift

Chih-Feng Wu

7

Multiplication of Signed Number

- Sequential multiplication of 2's-complement numbers with **right** shifts (Positive multiplier)

- Negative multiplicand
- Positive multiplier
- Proper sign-extension

Check:
 -10×11
 $= -110$
 $= -512 +$
 $256 +$
 $128 +$
 $16 + 2$

a	1	0	1	1	0
x	0	1	0	1	1
$p^{(0)}$	0	0	0	0	0
$+x_0a$	1	0	1	1	0
$2p^{(1)}$	1	1	0	1	1
$p^{(1)}$	1	1	0	1	1
$+x_1a$	1	0	1	1	0
$2p^{(2)}$	1	1	0	0	0
$p^{(2)}$	1	1	0	0	0
$+x_2a$	0	0	0	0	0
$2p^{(3)}$	1	1	1	0	0
$p^{(3)}$	1	1	1	0	0
$+x_3a$	1	0	1	1	0
$2p^{(4)}$	1	1	0	0	1
$p^{(4)}$	1	1	0	0	1
$+x_4a$	0	0	0	0	0
$2p^{(5)}$	1	1	1	0	0
$p^{(5)}$	1	1	1	0	0

Chih-Feng Wu

8

Multiplication of Signed Number (cont'd)

- Sequential multiplication of 2's-complement numbers with right shifts (negative multiplier)

- Negative multiplicand
- Negative multiplier

- In last step (the sign bit)
 - Subtract rather than add

a	1	0	1	1	0
x	1	0	1	0	1
$p^{(0)}$	0	0	0	0	0
$+x_0a$	1	0	1	1	0
$2p^{(1)}$	1	1	0	1	1
$p^{(1)}$	1	1	0	1	1
$+x_1a$	0	0	0	0	0
$2p^{(2)}$	1	1	1	0	1
$p^{(2)}$	1	1	1	0	1
$+x_2a$	1	0	1	1	0
$2p^{(3)}$	1	1	0	0	1
$p^{(3)}$	1	1	0	0	1
$+x_3a$	0	0	0	0	0
$2p^{(4)}$	1	1	1	0	0
$p^{(4)}$	1	1	1	0	0
$+(-x_4a)$	0	1	0	1	0
$2p^{(5)}$	0	0	0	1	1
$p^{(5)}$	0	0	0	1	1

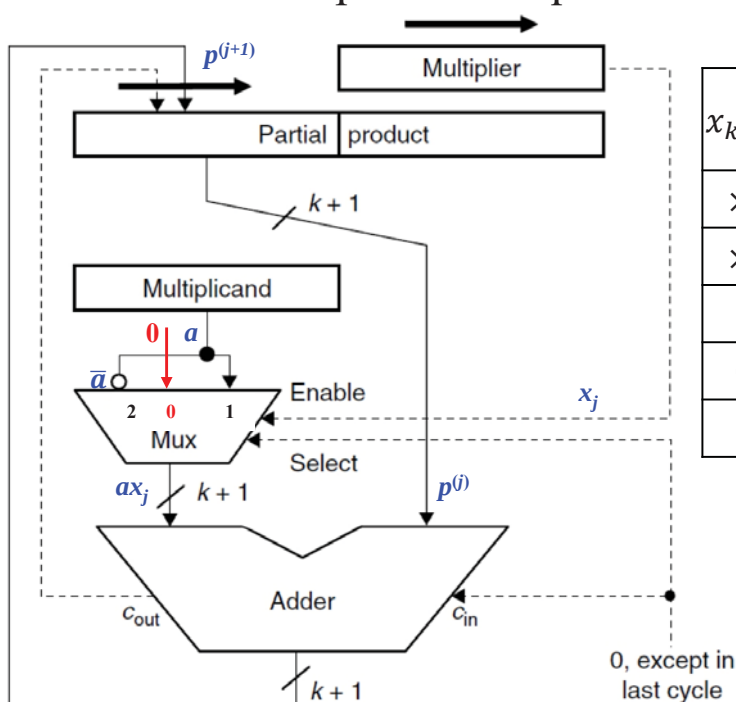
Check:
 -10×-11
 $= 110$
 $= 64 + 32 +$
 $8 + 4 + 2$

Chih-Feng Wu

9

Multiplication of Signed Number (cont'd)

- The 2's-complement sequential hardware multiplier



x_{k-1}	x_j ($j \neq k-1$)	MUX SEL	MUX O/P	c_{in}
$\times$	0	0	0	0
$\times$	1	1	a	0
$j = k-1$				
0	$\times$	0	0	0
1	$\times$	2	$\bar{a}$	1

Chih-Feng Wu

10



Booth's Recoding

Radix-2 Booth's recoding

- x_i : Encoding bit
- x_{i-1} : Reference bit
- y_i : Coded bit

$$y_i = x_{i-1} - x_i$$

x_i	x_{i-1}	y_i	Explanation
0	0	0	No string of 1s in sight
0	1	1	End of string of 1s in x
1	0	$\bar{1}$	Beginning of string of 1s in x
1	1	0	Continuation of string of 1s in x

A large number of consecutive 1s can be simplified based on the following fact.

- $2^j + 2^{j-1} + \dots + 2^{i+1} + 2^i = 2^{j+1} - 2^i$
- For example, $1111_2 = 1000\bar{1}_2 (= 2^4 - 2^0)$
 $15_d = 16_d - 1_d$
- Example:

$$\begin{array}{r} 1001 \ 1101 \ 1010 \ 1110 \\ (1) \ \bar{1}010 \ 0\bar{1}10 \ \bar{1}1\bar{1}1 \ 00\bar{1}0 \end{array}$$

Chih-Feng Wu

11



Carry-Save Adder (cont'd)

Sequential multiplication of 2's-complement numbers with right shifts by means of Booth's recoding

a	1	0	1	1	0
x	1	0	1	0	1
y	-1	1	-1	1	-1
Multiplier Booth-recoded					
$p^{(0)}$	0	0	0	0	0
$+y_0a$	0	1	0	1	0
$2p^{(1)}$	0	0	1	0	1
$p^{(1)}$	0	0	1	0	1
$+y_1a$	1	0	1	1	0
$2p^{(2)}$	1	1	1	0	1
$p^{(2)}$	1	1	1	0	1
$+y_2a$	0	1	0	1	0
$2p^{(3)}$	0	0	0	1	1
$p^{(3)}$	0	0	0	1	1
$+y_3a$	1	0	1	1	0
$2p^{(4)}$	1	1	1	0	0
$p^{(4)}$	1	1	1	0	0
$+y_4a$	0	1	0	1	0
$2p^{(5)}$	0	0	0	1	1
$p^{(5)}$	0	0	0	1	1

Check:

$$\begin{aligned} & -10 \times -11 \\ & = 110 \end{aligned}$$

$$\begin{aligned} & = 64 + 32 + \\ & \quad 8 + 4 + 2 \end{aligned}$$

Chih-Feng Wu

12



Multiplication by Constants

- ❑ Hardware aspects:
 - Synthesize special-purpose units such as filters
 - $y[t] = a_0x[t] + a_1x[t - 1] + a_2x[t - 2] + b_1y[t - 1] + b_2y[t - 2]$
- ❑ Constant coefficients: a_0, a_1, a_2, b_1, b_2
 - Find an easily expression (non-zero bit numbers)

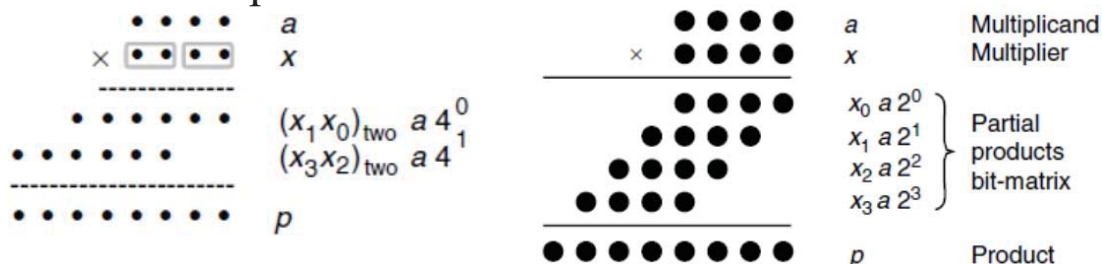


Preview of Fast Multipliers

- ❑ Multiplication as multi-operand addition problem
- ❑ Two ways to speed it up
 - Reducing the number of operands to be added
 - Handling more than one multiplier bit at a time (high-radix multipliers)
 - Adding the operands faster
 - Parallel/pipelined multi-operands addition (tree and array multipliers)

Radix-4 Multiplier

Radix-4 Multiplication



Multiplication with right shifts in radix- r

$$\blacksquare p^{(j+1)} = (p^{(j)} + x_j a r^k) r^{-1}$$

|—add—|
|—shift right—|

$$\blacksquare p^{(0)} = 0 \quad \text{and} \quad p^{(k)} = p$$

$$\blacksquare \text{After } k \text{ iterations: } p^{(k)} = p = ax + p^{(0)} r^{-k}$$

Radix-4 Multiplier (cont'd)

Multiplication with left shifts in radix- r

$$\blacksquare p^{(j+1)} = r p^{(j)} + x_{k-j-1} a$$

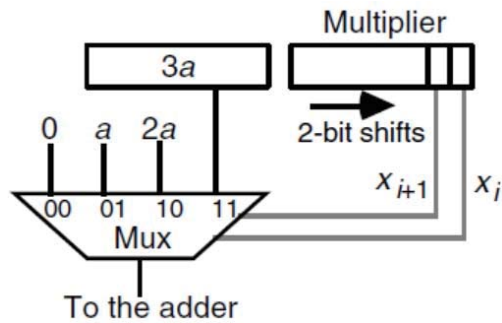
|shift left|
|—add—|

$$\blacksquare p^{(0)} = 0 \quad \text{and} \quad p^{(k)} = p$$

$$\blacksquare \text{After } k \text{ iterations: } p^{(k)} = p = ax + p^{(0)} r^k$$

Radix-4 Multiplier (cont'd)

- The multiple generation part of a radix-4 multiplier with precomputation of $3a$
 - Without overlap on multiplier



=====									
a				0	1	1	0		
$3a$				0	1	0	0	1	0
x				1	1	1	0		
=====									
$p^{(0)}$				0	0	0	0		
$+(x_1x_0)_{\text{two}} a$				0	0	1	1	0	0

$4p^{(1)}$				0	0	1	1	0	0
$p^{(1)}$						0	0	1	1
$+(x_3x_2)_{\text{two}} a$				0	1	0	0	1	0

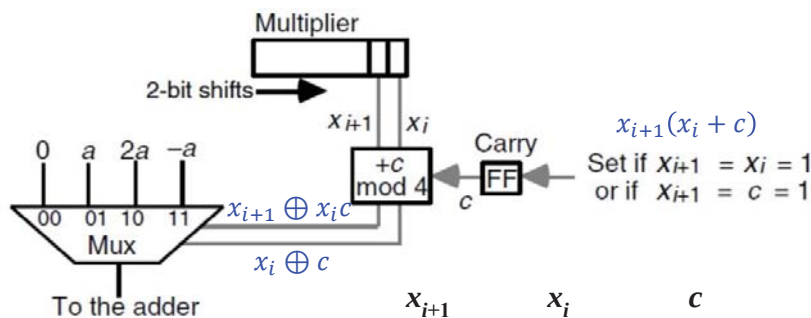
$4p^{(2)}$				0	1	0	1	0	1
$p^{(2)}$						0	1	0	1
=====									

Chih-Feng Wu

17

Radix-4 Multiplier (cont'd)

- The multiple generation part of a radix-4 multiplier based on replacing $3a$ with $4a$ (carry into next higher radix-4 multiplier digit) and $-a x_i \oplus c$ (without overlap on multiplier)



x_{i+1}	x_i	c	Mux control		Set carry
0	0	0	0	0 (0)	0
0	0	1	0	1 (a)	0
0	1	0	1	1 (a)	0
0	1	1	1	0 (2a)	0
1	0	0	1	0 (2a)	0
1	0	1	1	1 (-a)	1 (4a)
1	1	0	1	1 (-a)	1 (4a)
1	1	1	0	0 (0)	1 (4a)

Chih-Feng Wu

18

Modified Booth's Recoding

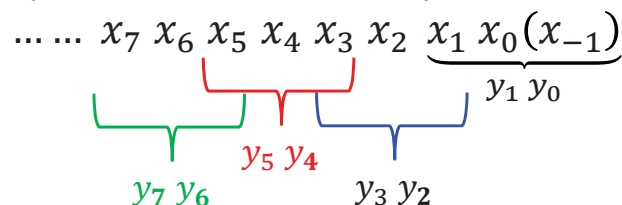
- ❑ Booth's **radix-2** recoding
 - Handle 2's complement multiplier
 - Add a zero to the **left** of the multiplier ($x_k = 0$ for k -bit multiplier) to ensure the correct result for the unsigned numbers
- ❑ Drawbacks for Booth's algorithm
 - 1st: Inconvenient for designing a synchronous multiplier because the number of add/subtract operations is variable.
 - 2nd: Inefficient operations when there are isolated 1's, for instance, 001010101(0) is recoded as 01 $\bar{1}$ 1 $\bar{1}$ 1 $\bar{1}$ 1 $\bar{1}$ requiring eight operations.
- ❑ Improvement by **examining 3-bit** of multiplier (**radix-4**) at a time

Chih-Feng Wu

19

Modified Booth's Recoding (cont'd)

- ❑ Modified Booth's Recoding (Radix-4)
 - x_{i+1} and x_i with a reference bit x_{i-1} recoded into to y_{i+1} and y_i



- The **radix-4** $z_{i/2} = (y_{i+1}, y_i)_{two}$, $i \in even$
 - Obtain directly from x_{i+1} and x_i without the radix-2 recoded from y_{i+1} and y_i
 - Booth's Recoding (Radix-2)

x_i	x_{i-1}	y_i	Explanation
0	0	0	No string of 1s in sight
0	1	1	End of string of 1s in x
1	0	$\bar{1}$	Beginning of string of 1s in x
1	1	0	Continuation of string of 1s in x

Chih-Feng Wu

20

Modified Booth's Recoding (cont'd)

Modified Booth's Recoding (a: Multiplicand)

x_{i+1}	x_i	x_{i-1}	y_{i+1}	y_i	$z_{i/2}$	Explanation
0	0	0	0	0	0	No string of 1s in sight
0	0	1	0	1	a	End of string of 1s
0	1	0	1	1	a	Isolated 1
0	1	1	1	0	2a	End of string of 1s
1	0	0	1	0	-2a	Beginning of string of 1s
1	0	1	1	1	-a	End a string, begin new one
1	1	0	0	1	-a	Beginning of string of 1s
1	1	1	0	0	0	Continuation of string of 1s

Booth-encoded digit (radix-4)

- $z_{i/2} = (x_i + x_{i-1}) - 2x_{i+1}$
- $z_{i/2} \in [-2, 2]$

x_i	x_{i-1}	y_i	Explanation
0	0	0	No string of 1s in sight
0	1	1	End of string of 1s in x
1	0	1	Beginning of string of 1s in x
1	1	0	Continuation of string of 1s in x

Chih-Feng Wu

21

Modified Booth's Recoding (cont'd)

Example of radix-4 multiplication with modified Booth's recoding of the 2's-complement multiplier

a	a	0 1 1 0
x	x	1 0 1 0
z	z	-1 -2
		Radix-4 recoded version of x
$p^{(1)} = z_0 a$	$p^{(0)}$	0 0 0 0 0 0
$p^{(2)} = z_1 a$	$+z_0 a$	1 1 0 1 0 0
	$4p^{(1)}$	1 1 0 1 0 0
	$p^{(1)}$	1 1 1 1 0 1
	$+z_1 a$	1 1 1 0 1 0
	$4p^{(2)}$	1 1 0 1 1 1
	$p^{(2)}$	1 1 0 1 1 1 0 0

Unsigned number (k-bit)

- No. of Booth-encoded digits : $\lceil k/2 \rceil + 1 = \lceil (k+1)/2 \rceil$
- $x_{k-1} = 0$, and $x_k = x_{k+1} = 0$ for MSB = 1

2's complement (k-bit)

- No. of Booth-encoded digits: $\lceil k/2 \rceil$ digit
- $x_k = x_{k-1}$ (signed-extension) for $k \in \text{odd}$, and $x_{-1} = 0$

Chih-Feng Wu

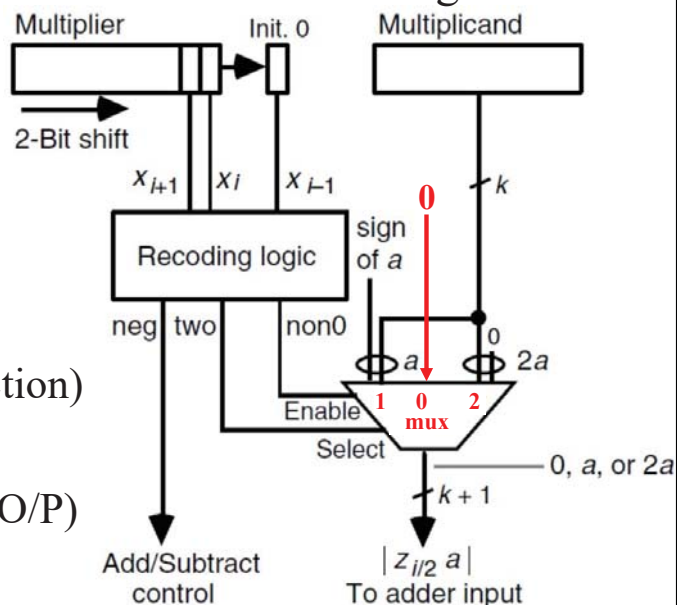
22

Modified Booth's Recoding (cont'd)

Multiple Generation with Radix-4 Booth's Recoding

Digit	neg	two	non0	mux SEL
-2	1	1	0	2
-1	1	0	1	1
0	0	0	0	0
1	0	0	1	1
2	0	1	0	2

- neg: 0 (addition), 1 (subtraction)
- non0: a non-zero ("a" O/P)
- two: a non-zero of 2 ("2a" O/P)

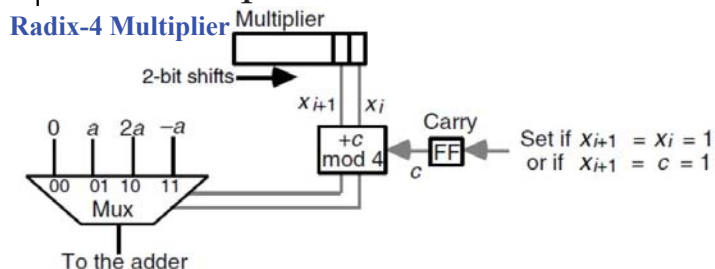


Chih-Feng Wu

23

Modified Booth's Recoding (cont'd)

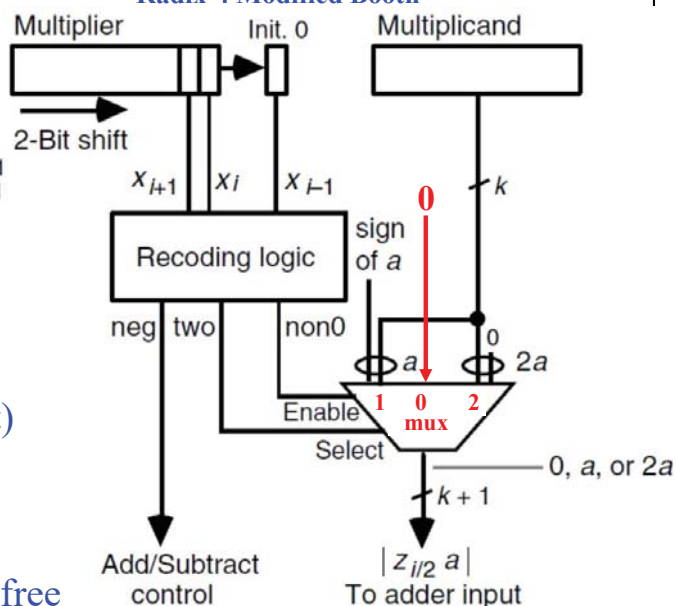
Comparison



- Radix-4 Multiplier
 - Serial operation (right to left)
 - Delay time (_____)

- Radix-4 Modified Booth
 - Parallel operation and carry-free
 - Apply to the design of tree and array multiplication
 - Delay time (_____)

Radix-4 Modified Booth



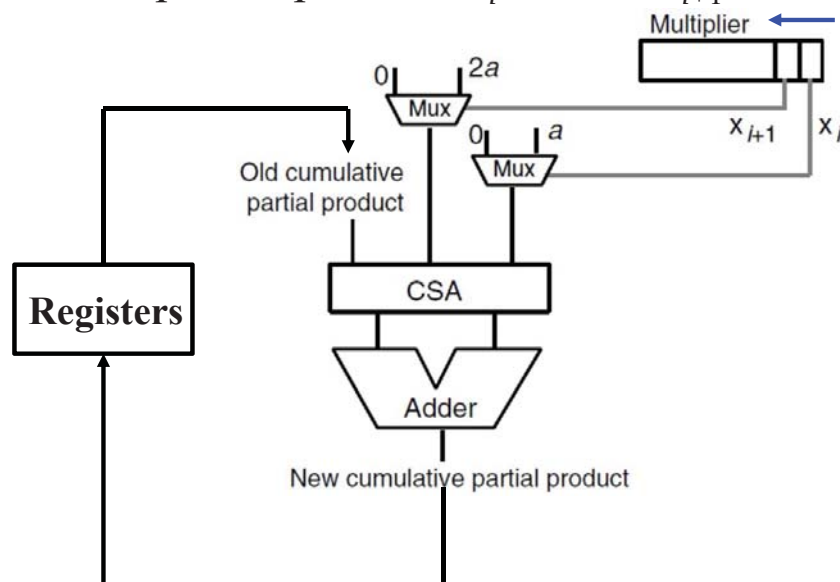
Chih-Feng Wu

24

Using Carry-Save Adder

- Radix-4 multiplication with a carry-save adder used to combine the cumulative partial product, $x_i a$, and $2x_{i+1}a$ into two numbers

■ $3a = 2a + a$

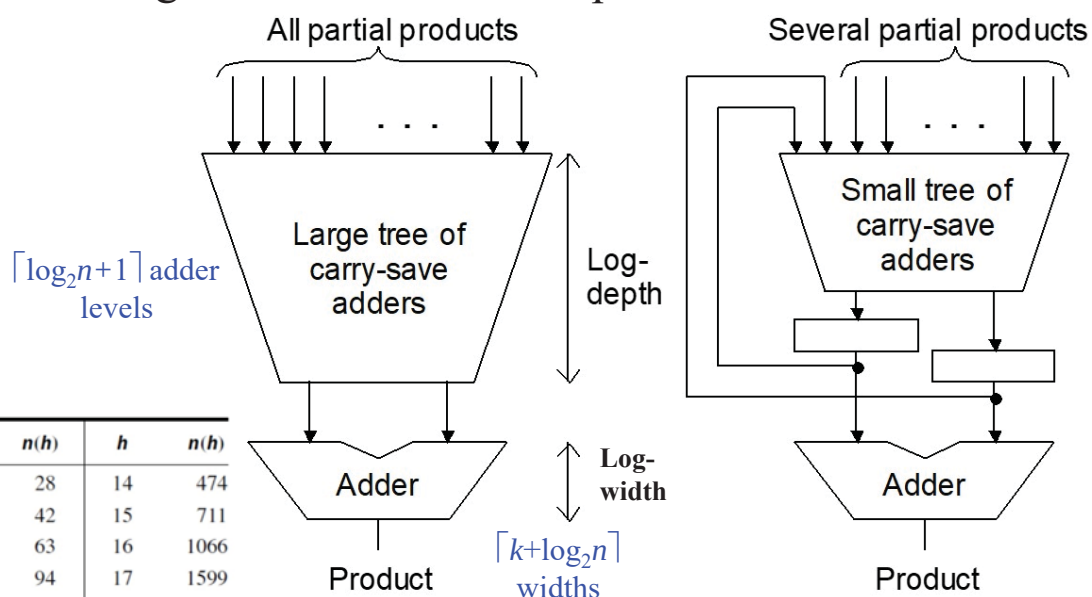


Chih-Feng Wu

25

Full-Tree versus Partial-Tree Multiplier

- Schematic diagrams for full-tree and partial-tree multipliers



h	$n(h)$	h	$n(h)$	h	$n(h)$
0	2	7	28	14	474
1	3	8	42	15	711
2	4	9	63	16	1066
3	6	10	94	17	1599
4	9	11	141	18	2398
5	13	12	211	19	3597
6	19	13	316	20	5395

Chih-Feng Wu

26

Full-Tree Multiplier Design

□ Distinguished in three elements:

■ 1. Multiple-forming circuits

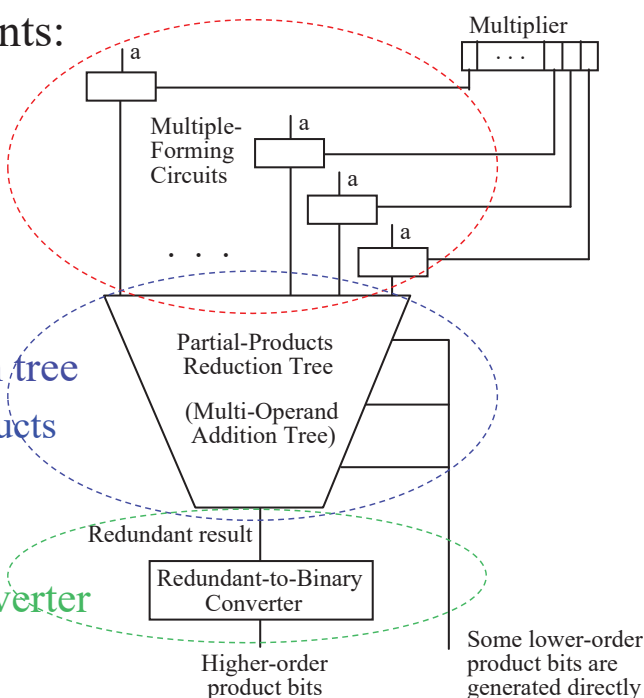
- A collection of AND gates
 - Binary multiplier
 - Radix-4 Booth's
 - ...

■ 2. Partial products reduction tree

- Combinational partial products
- CSA tree

■ 3. Redundant-to-binary converter

- CPA

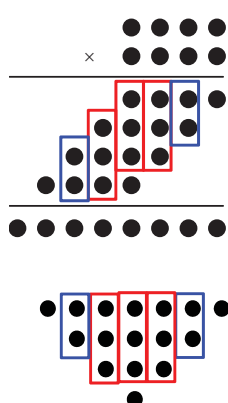


Chih-Feng Wu

27

Full-Tree Multiplier Design (cont'd)

□ Two different binary 4×4 tree multipliers



Wallace Tree
(5 FAs + 3 HAs + 4-Bit Adder)

1	2	3	4	3	2	1
		FA	FA	FA	HA	

1	3	2	3	2	1	1
		FA	HA	FA	HA	

2	2	2	2	1	1	1
				4-Bit Adder		

1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---

Dadda Tree
(4 FAs + 2 HAs + 6-Bit Adder)

1	2	3	4	3	2	1
		FA	FA			

1	3	2	2	3	2	1
		FA	HA	HA	FA	

2	2	2	2	1	2	1
				6-Bit Adder		

1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---

■ Wallace tree

- Combine the partial-product **at the earliest stage** for building CSA tree
- The **fastest** possible design

■ Dadda tree

- Combine the partial-product **as late as possible**
- The Simpler CSA tree and a **wider** CPA

Chih-Feng Wu

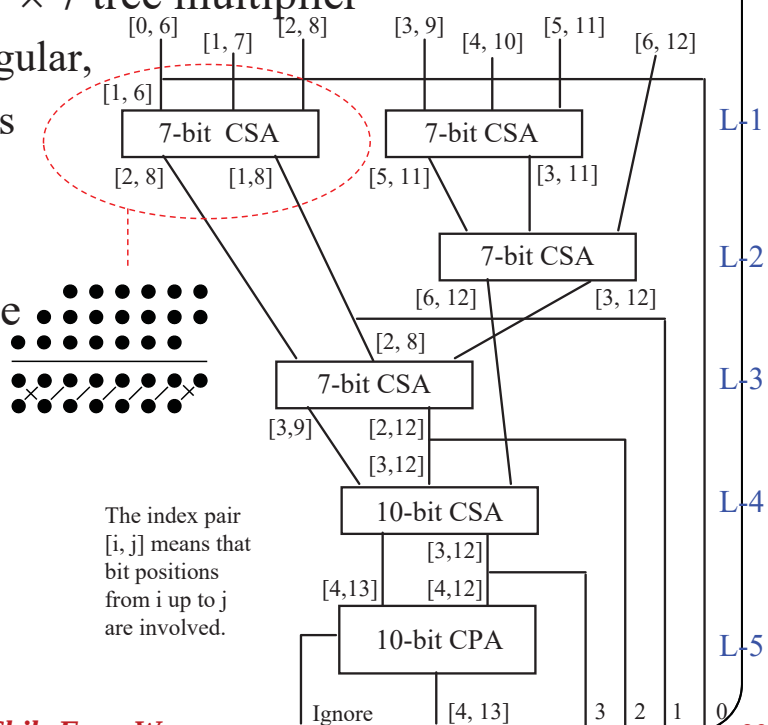
28

Full-Tree Multiplier Design (cont'd)

□ Possible CSA tree for a 7×7 tree multiplier

- CSA trees are quite irregular, causing some difficulties in VLSI realization.

- Do we have alternative methods for partial products reduction?



Chih-Feng Wu

29

Alternative Reduction Tree

□ Based on Table 8.1

- An $(11; 2)$ -counter: at least 5 FA levels

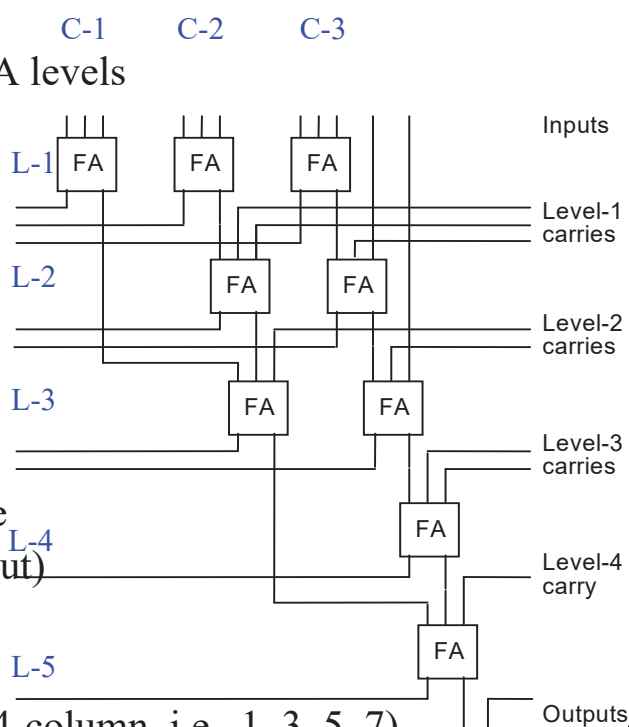
h	$n(h)$	h	$n(h)$	h	$n(h)$
0	2	7	28	14	474
1	3	8	42	15	711
2	4	9	63	16	1066
3	6	10	94	17	1599
4	9	11	141	18	2398
5	13	12	211	19	3597
6	19	13	316	20	5395

- All outputs generated after the same number of FA delays.

- Balanced-delay tree are suitable for VLSI implementation (layout) of parallel multiplier.

- 3-column (1, 3, 5)

- Easy to expand from 11 to 18 (4-column, i.e., 1, 3, 5, 7)



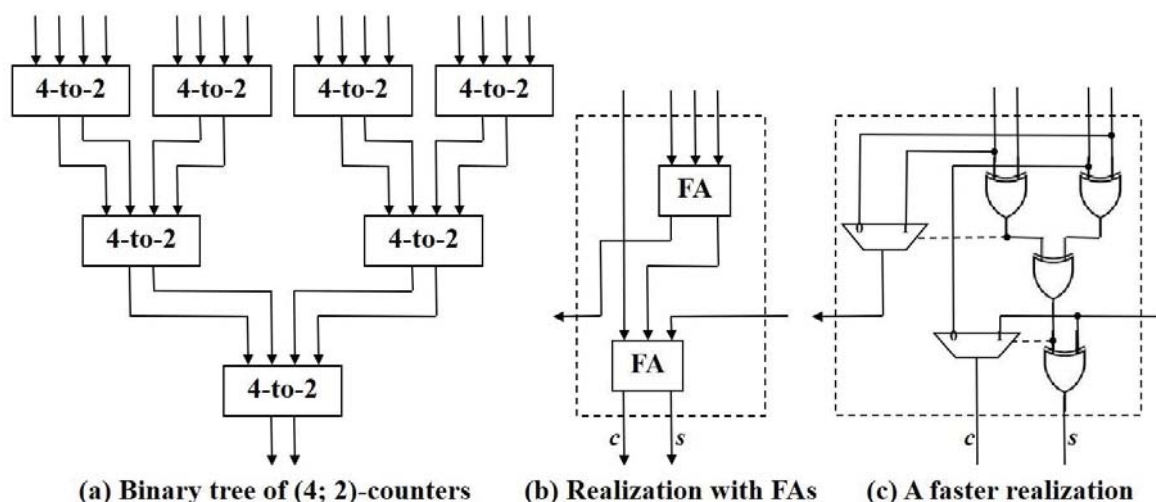
Chih-Feng Wu

30

Binary Tree of 4-to-2 Reduction Module

□ Basic building block (4; 2) reduction module

- A binary tree is **more regular** than a 3-to-2 reduction tree when laid out in VLSI.



$4t_{xor}$
Chih-Feng Wu

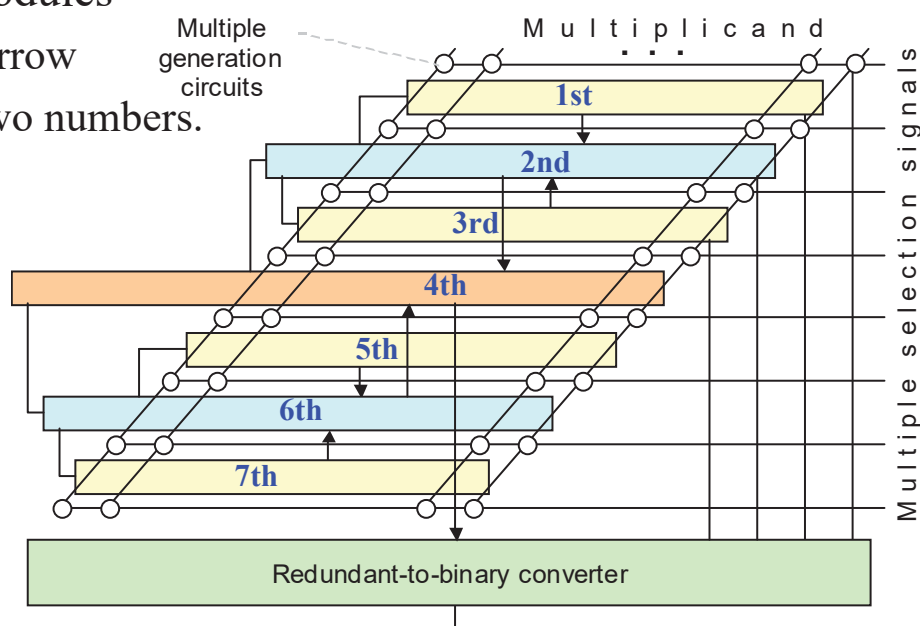
$3t_{xor}$

31

Binary Tree of 4-to-2 Reduction Module (cont'd)

□ Layout of a partial-products reduction tree composed of 4-to-2 reduction modules

- Each solid arrow represents two numbers.



Chih-Feng Wu

32

Tree Multiplier for Signed Number

- ❑ Multiplying 2's-complement number
 - Signed-number addition for all partial products
 - Sign-extension for all partial products
- ❑ Sign-extension to the width of the final result
 - Extended positions -- Sign -- Magnitude positions -----

MSB

x_{k-1}	x_{k-1}	x_{k-1}	x_{k-1}	x_{k-1}	x_{k-1}	x_{k-2}	x_{k-3}	x_{k-4}	$\dots$
y_{k-1}	y_{k-1}	y_{k-1}	y_{k-1}	y_{k-1}	y_{k-1}	y_{k-2}	y_{k-3}	y_{k-4}	$\dots$
z_{k-1}	z_{k-1}	z_{k-1}	z_{k-1}	z_{k-1}	z_{k-1}	z_{k-2}	z_{k-3}	z_{k-4}	$\dots$

- ❑ Sharing of FAs to reduce the CSA width in a signed tree multiplier



Chih-Feng Wu

33

Array Multiplier

- ❑ Array multiplier
 - A slow multiplier
 - A regular structure with the short wire connections (simple and efficient layout)
- ❑ A simple expression of an array multiplier can be described as

$$P = \{p_7, p_6, \dots, p_1, p_0\} \text{ and } p_i = \sum_{j+k=i} a_j b_k + c_{i-1}$$

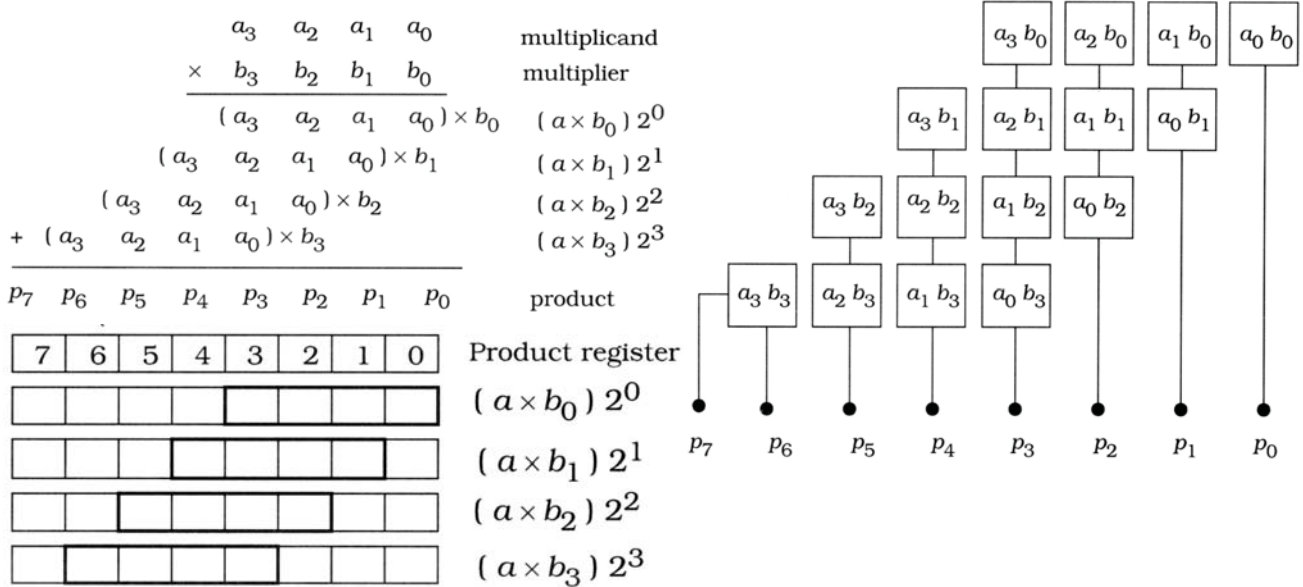
- j, k : the bit widths of multiplicand and multiplier
- i : the product width of the multiplier
- The product bits are obtained by summing each of the i th columns, accounting for a carry from the $(i-1)$ th column.

Chih-Feng Wu

34

Array Multiplier (cont'd)

4 x 4 Multiplication



Chih-Feng Wu

35

Array Multiplier (cont'd)

Basic cell for Array Multiplier

Bit-level multiplication

- $a_i \times b_i = a_i b_i \rightarrow$ AND gate operation

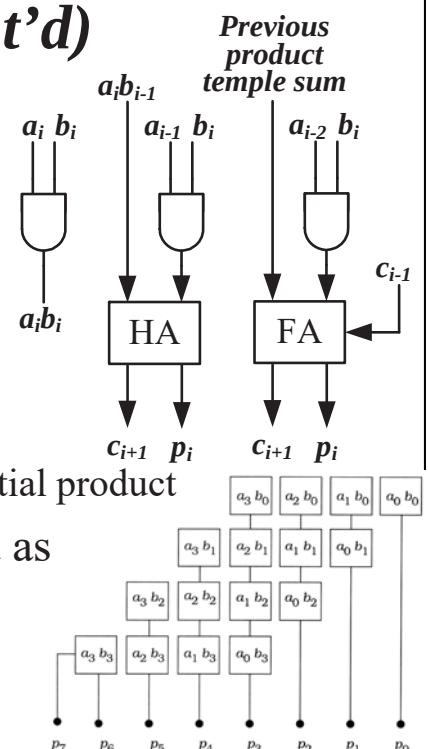
FA

- One bit for new partial product $a_i \cdot b_i$
- One bit for the previously accumulated partial product

The equivalent description can be derived as

$$A = \sum_{j=0}^{n-1} a_j 2^j \text{ and } B = \sum_{k=0}^{n-1} b_k 2^k$$

$$P = AB = \left(\sum_{j=0}^{n-1} a_j 2^j \right) \left(\sum_{k=0}^{n-1} b_k 2^k \right) = \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} a_j b_k 2^{j+k}$$

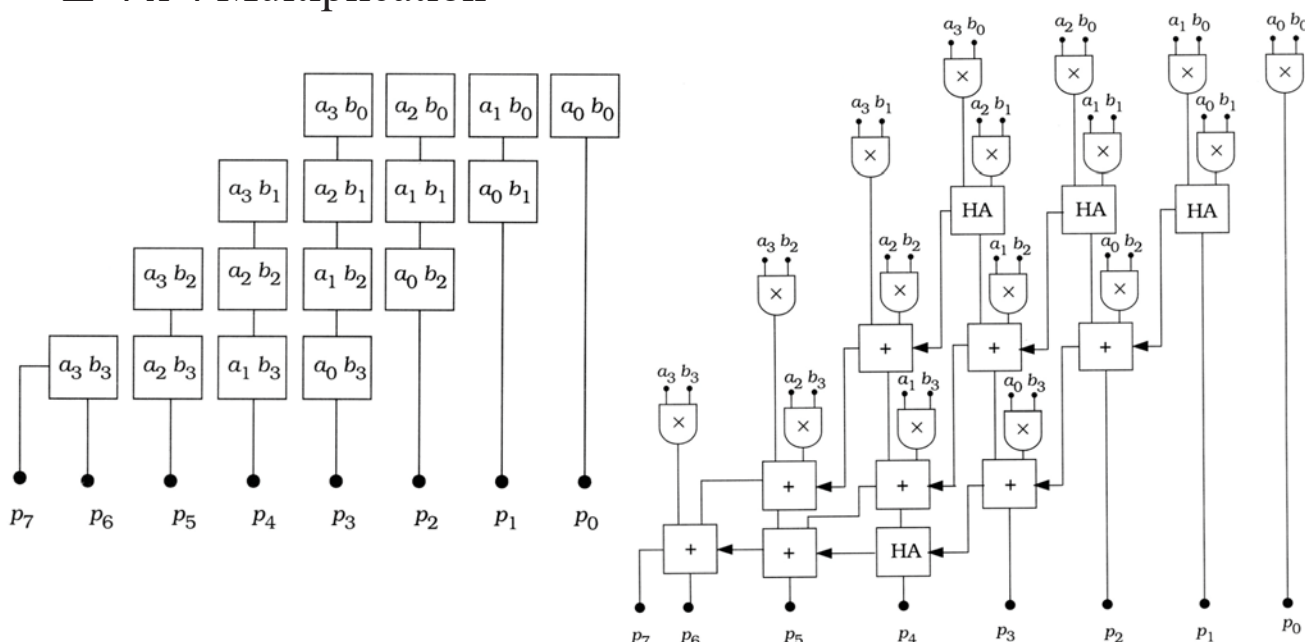


Chih-Feng Wu

36

Array Multiplier (cont'd)

4 x 4 Multiplication



Chih-Feng Wu

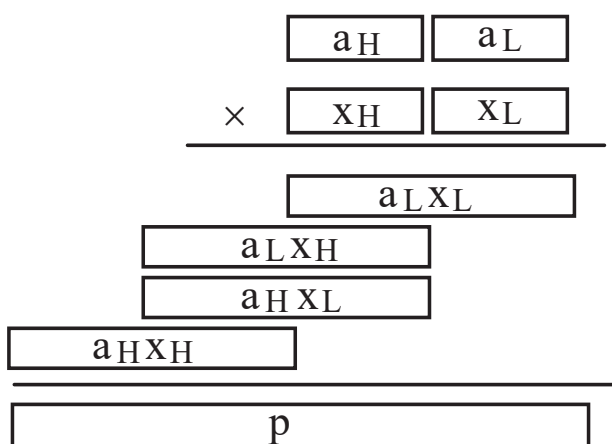
37

Divide-and-Conquer Multiplier

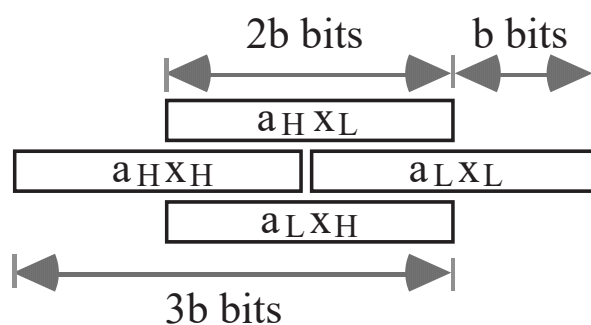
Building wide multiplier from narrower ones

Establish a $2b \times 2b$ multiplier based-on a $b \times b$ multiplier

- Use 4 ($b \times b$) multipliers to compute 4 partial products, a_H, a_L, x_H , and x_L



Rearranged partial products in $2b$ -by- $2b$ multiplication

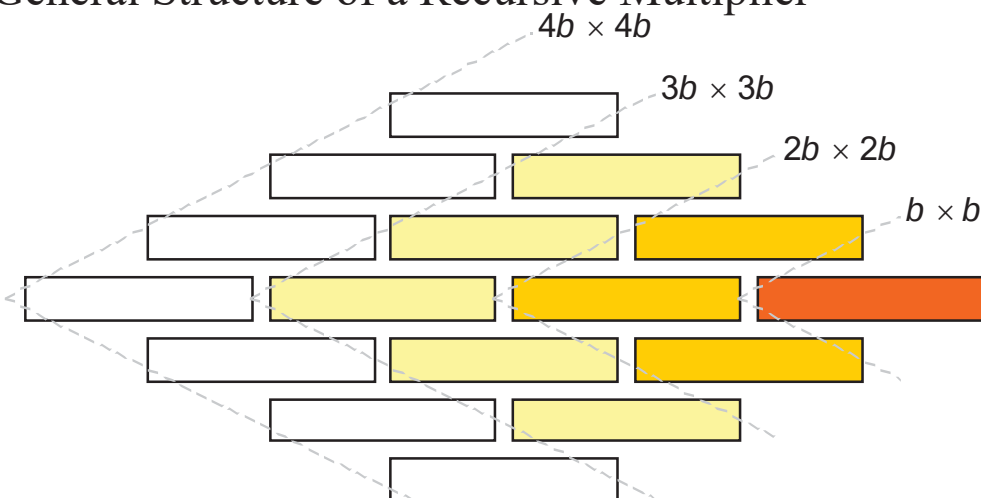


Chih-Feng Wu

38

Divide-and-Conquer Multiplier (cont'd)

□ General Structure of a Recursive Multiplier



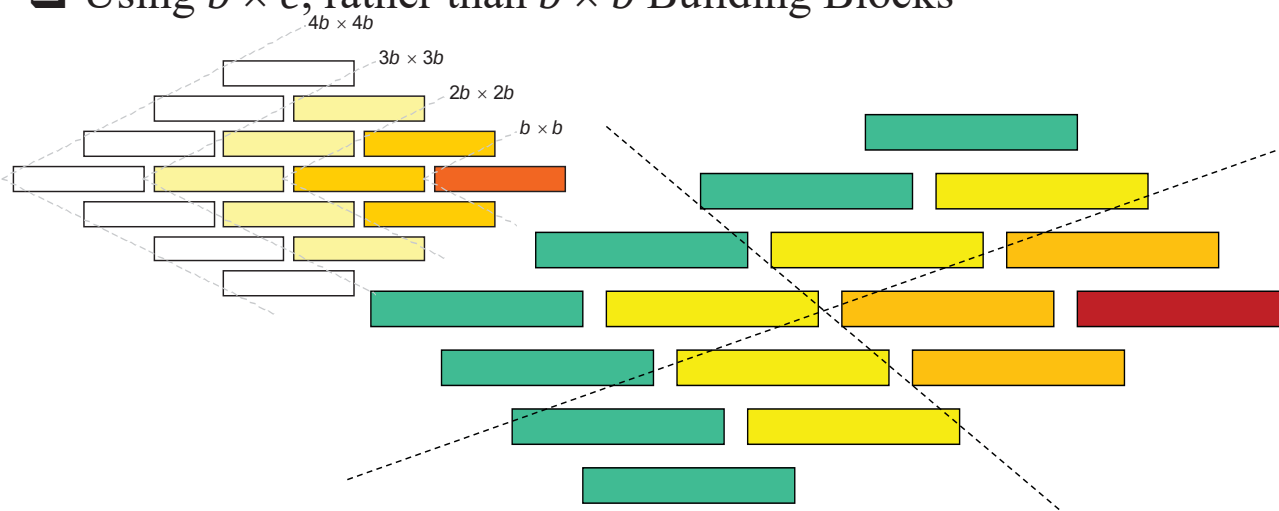
- $2b \times 2b$: use (3; 2)-counters
- $3b \times 3b$: use (5; 2)-counters
- $4b \times 4b$: use (7; 2)-counters

Chih-Feng Wu

39

Divide-and-Conquer Multiplier (cont'd)

□ Using $b \times c$, rather than $b \times b$ Building Blocks



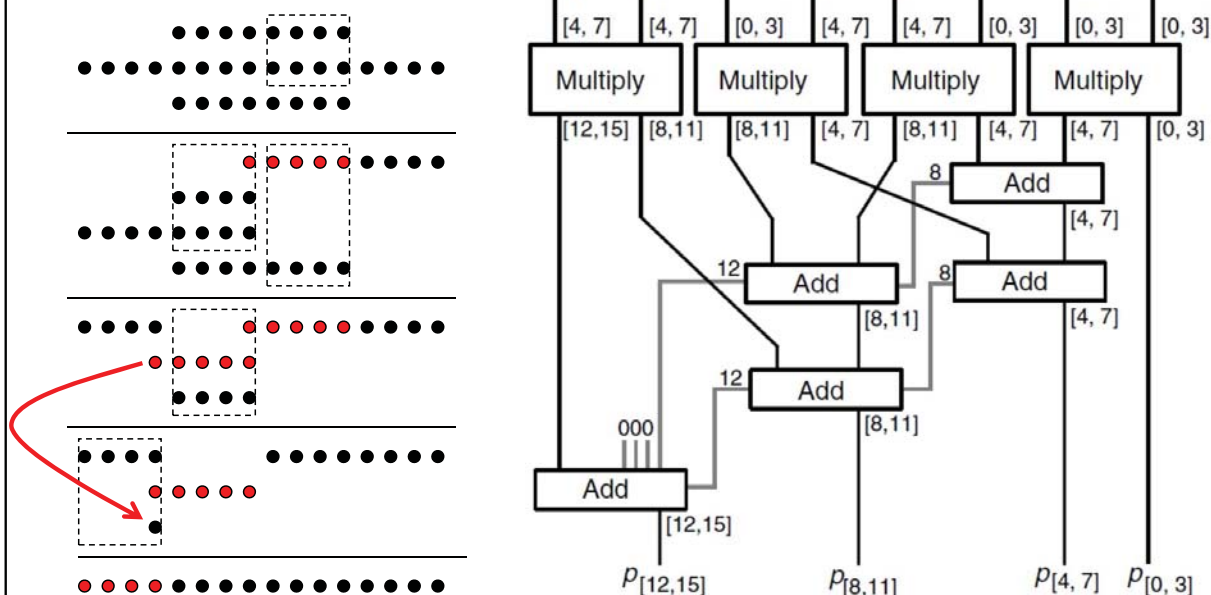
- $2b \times 2c$: use $b \times c$ multipliers and (3; 2)-counters
- $2b \times 4c$: use $b \times c$ multipliers and (5; 2)-counters
- $gb \times hc$: use $b \times c$ multipliers and (?; 2)-counters

Chih-Feng Wu

40

Divide-and-Conquer Multiplier (cont'd)

- Using 4×4 multipliers and 4-bit adders to synthesize an 8×8 multiplier



Chih-Feng Wu

41

Divide-and-Conquer Multiplier (cont'd)

- $2b \times 2b$ multiplication requires four $b \times b$ multiplications

$$(2^b a_H + a_L) \times (2^b x_H + x_L) = 2^{2b} a_H x_H + 2^b (a_H x_L + a_L x_H) + a_L x_L$$

- 4 multiplications and 3 additions

- Karatsuba Multiplication

- One of the four multiplications can be removed at the expense of introducing a few additions.

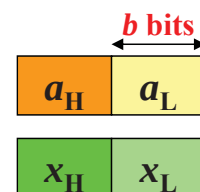
$$(2^b a_H + a_L) \times (2^b x_H + x_L) = 2^{2b} a_H x_H + 2^b [(a_H + a_L) \times (x_H + x_L)] - a_H x_H - a_L x_L + a_L x_L$$

Mult 1

Mult 3

Mult 2

- 3 multiplications and 6 additions



Chih-Feng Wu

42